

ALMA MATER STUDIORUM - UNIVERSITÀ DI BOLOGNA

Statistical Sciences Department

Second Cycle Degree in Quantitative Finance



COMPUTATIONAL FINANCE

March 2026

Martingale Verification under the Heston Stochastic Volatility Model

Fourier pricing, quadratic-exponential simulation,
and vectorized pathwise repricing

Mahed Ali

mahed.ali@studio.unibo.it

Course Instructor: Prof. Pietro Rossi

Department of Statistical Sciences “Paolo Fortunati”

Contents

1	Introduction	2
2	Model and mathematical framework	2
2.1	Heston dynamics	2
2.2	Pricing identities	3
2.3	Martingale property	4
2.4	Monte Carlo estimators	4
3	Pricing methodology and code logic	4
3.1	Time-0 Fourier Pricing	5
3.2	Variance Simulation: Quadratic-Exponential Scheme	5
3.3	Spot Price Simulation: Exact Conditional Scheme	6
3.4	Vectorized Pathwise Fourier Repricing	6
4	Numerical results and discussion	7
4.1	Time-0 benchmark prices	7
4.2	Internal consistency checks	7
4.3	Monthly martingale diagnostics for Stock, Put and Call	8
4.4	Monthly martingale diagnostics for digital and asset-or-nothing claims	8
4.5	Graphical martingale verification	9
4.6	Maximum deviation analysis	10
4.7	Computational cost	10
4.8	Sensitivity to the Fourier cutoff	11
5	Interpretation of the results	12
6	Limitations and possible extensions	12
7	Conclusion	13
A	Module Layout	14
A.1	Execution Instructions and Output Files	14

1 Introduction

This project implements a Fourier-based pricing engine for the Heston stochastic volatility model, computes the time-0 prices of six European-type claims, simulates monthly trajectories of the joint stock-variance state up to $T = 1$, and verifies numerically that the martingale property holds for both the stock price and the continuation value of each contract under the Heston model. The six claims considered are:

$$\begin{aligned} \mathcal{C}(S_0, T) &= \mathbb{E}[(S_T - K)^+] & \mathcal{P}(S_0, T) &= \mathbb{E}[(K - S_T)^+] \\ \mathcal{P}_{cn}(S_0, T) &= \mathbb{E}[\mathbf{1}_{[K > S_T]}] & \mathcal{P}_{an}(S_0, T) &= \mathbb{E}[S_T \mathbf{1}_{[K > S_T]}] \\ \mathcal{C}_{cn}(S_0, T) &= \mathbb{E}[\mathbf{1}_{[K < S_T]}] & \mathcal{C}_{an}(S_0, T) &= \mathbb{E}[S_T \mathbf{1}_{[K < S_T]}] \end{aligned}$$

For each month t_j , $j = 1, \dots, 11$, the numerical verification for every option $\mathcal{O}$ listed above is

$$\frac{1}{N} \sum_n S^n(t_j) \approx S_0, \quad \frac{1}{N} \sum_n \mathcal{O}(S^n(t_j), T - t_j) \approx \mathcal{O}(S_0, T)$$

The report compares intermediate-date continuation values with the corresponding time-0 benchmarks. The purpose of the report is therefore threefold: (i). state the model and the pricing identities used; (ii). explain the numerical methodology implemented in the code; and (iii). report and interpret the numerical and graphical results.

2 Model and mathematical framework

2.1 Heston dynamics

The Heston model is defined by the set of SDEs [1]:

$$\begin{aligned} S_t &= S_0 e^{X_t} \\ dX_t &= r dt - \frac{\nu_t}{2} dt + \sqrt{\nu_t} dW_t \quad X_0 = 0 \\ d\nu_t &= \lambda(\bar{\nu} - \nu_t) dt + \eta \sqrt{\nu_t} dY_t \quad \nu_0 = \nu_0 \\ \text{Cov}(dW_t dY_t) &= \rho dt \end{aligned}$$

The variance process ν_t is a square-root diffusion of CIR type [2]. The parameter λ controls the speed of mean reversion of the variance toward its long-run level $\bar{\nu}$, the parameter η determines the volatility of the variance process itself, ν_0 is the initial variance, and ρ is the instantaneous correlation between the Brownian motion driving the stock and the Brownian motion driving the variance. The interest rate and dividend yield are both taken equal to zero, i.e., $r = q = 0$, so no discounting adjustment is needed in the martingale verification, and the stock price process itself must be a martingale under the pricing measure. An important structural feature of the Heston model is that the relevant state variable at time t is not only the stock price S_t , but the pair (S_t, ν_t) . Hence, intermediate-date continuation values must be evaluated from the full state (S_t, ν_t) , not from S_t alone. An important parameter constraint is the Feller condition [3], which requires $2\lambda\bar{\nu} > \eta^2$, equivalently $(2\lambda\bar{\nu})/\eta^2 > 1$. Under the given parameters, the Feller ratio evaluates to $(2\lambda\bar{\nu})/\eta^2 \approx 0.1614$, which indicates a violation. This means that the variance

process can reach zero during simulation, which has direct implications for the numerical scheme.

The parameter set and numerical settings used in the implementation are summarized in Table 1.

Table 1: Heston model parameters and numerical settings.

Quantity	Symbol	Value
Initial stock	S_0	1.2
Strike	K	1.2
Maturity	T	1.0
Mean reversion speed	λ	1.03179746
Long-run variance	$\bar{\nu}$	0.07819284
Volatility of variance	η	1.0
Initial variance	ν_0	0.04
Correlation	ρ	-0.6450755
Outer paths	$N = 2^{13}$	8192
Feller	$(2\lambda\bar{\nu})/\eta^2$	0.161358
Monitoring dates	n_m	12 monthly dates
Internal variance step	Δt	1/730 (twice daily)
Fourier cutoff	X_c	5.0
Random seed	–	29283

The monthly dates are given. However, the internal step $\Delta t = 1/730$ is a numerical choice used to simulate the variance process on a finer grid than a daily step. This improves the stability of the martingale tests without changing the economic problem. All results in this report are reproducible: the seed above is the script default, so re-running with no arguments reproduces every figure exactly.

2.2 Pricing identities

The six claims are not all independent. The two cash-or-nothing digitals partition the terminal event space, while the two asset-or-nothing claims split the terminal stock value. Therefore,

$$\mathcal{C}_{cn}(S_0, T) = 1 - \mathcal{P}_{cn}(S_0, T), \quad \mathcal{C}_{an}(S_0, T) = S_0 - \mathcal{P}_{an}(S_0, T)$$

Hence, it is sufficient to compute directly Put, Call, $\mathcal{P}_{cn}$, and $\mathcal{P}_{an}$; the complementary quantities $\mathcal{C}_{cn}$ and $\mathcal{C}_{an}$ follow from these identities. The vanilla put decomposes into digital building blocks:

$$(K - S_T)^+ = K \mathbf{1}_{[S_T < K]} - S_T \mathbf{1}_{[S_T < K]}$$

so that,

$$\mathcal{P}(S_0, T) = K \mathcal{P}_{cn}(S_0, T) - \mathcal{P}_{an}(S_0, T)$$

Since, $r = q = 0$ and $S_0 = K = 1.2$, put-call parity immediately gives $\mathcal{C}(S_0, T) = \mathcal{P}(S_0, T)$. These identities serve as built-in consistency checks for the pricing layer of the implementation. Under the Heston model, the conditional characteristic function of the log-price is affine in the current variance state ν_t . Specifically, for complex argument k :

$$\log \phi(k, \tau, \nu_t) = A(k, \tau) - \nu_t B(k, \tau)$$

where A and B are deterministic complex-valued functions of the Fourier argument k and remaining time τ , derived from the Riccati equations of the Heston model. This affine structure underlies both the time-0 Fourier pricing through the provided Fourier routine `ft_opt.py` and the vectorized pathwise repricing routine at intermediate dates.

2.3 Martingale property

Let

$$V^{\mathcal{O}}(t, S_t, \nu_t) = \mathbb{E}[\text{payoff}_{\mathcal{O}}(S_T) \mid \mathcal{F}_t]$$

denote the continuation value at time t of a generic claim $\mathcal{O}$. Since $r = q = 0$, for each claim the process $V^{\mathcal{O}}(t, S_t, \nu_t)$ must be a martingale. Therefore,

$$\mathbb{E}[V^{\mathcal{O}}(t, S_t, \nu_t)] = V^{\mathcal{O}}(0, S_0, \nu_0) = \mathcal{O}(S_0, T) \quad \text{for all } t \leq T$$

$$\mathbb{E}[S_t] = S_0$$

Because volatility is stochastic, the current variance state ν_t is part of the Markov state of the model. For this reason, the correct intermediate pricing object is $V^{\mathcal{O}}(t, S_t, \nu_t)$, rather than a reduced expression depending only on S_t and the residual maturity.

2.4 Monte Carlo estimators

For a generic quantity $Z_1, \dots, Z_N$, the Monte Carlo estimator of its expectation and the corresponding standard error is

$$\hat{\mu}_N = \frac{1}{N} \sum_{n=1}^N Z_n$$

$$\text{SE}(\hat{\mu}_N) = \frac{\hat{\sigma}_N}{\sqrt{N}}$$

where

$$\hat{\sigma}_N^2 = \frac{1}{N} \sum_{n=1}^N (Z_n - \hat{\mu}_N)^2$$

These estimators are applied at each monthly monitoring date to both the simulated stock values and the repriced continuation values of all six claims. The martingale property is considered numerically verified when the benchmark value remains close to the Monte Carlo mean.

3 Pricing methodology and code logic

The implementation is organized as a single Python script `heston_martingales.py` that draws on the CFLib library. The computation is divided into four main stages:

Step 1: Input parsing and model construction. The code reads the parameters, verifies that the implementation is used in the setting $r = q = 0$, fixes the random seed for reproducibility, and constructs the Heston model object with the prescribed parameters.

Step 2: Time-0 pricing of all claims. The function `option_prices_t0` calls the Fourier pricing routine `ft_opt` to compute Put, Call, $\mathcal{P}_{cn}$, and $\mathcal{P}_{an}$. The remaining quantities $\mathcal{C}_{cn}$ and $\mathcal{C}_{an}$ are then recovered from the algebraic identities in Section 2.2.

Step 3: Simulation of monthly outer states. The function `simulate_outer_paths` builds the monthly monitoring grid $t_j = j/12$, $j = 0, 1, \dots, 12$, simulates the variance process using the Quadratic-Exponential (QE) CIR scheme, and then generates the stock trajectories with `mc_heston`. This produces the outer states $(S^n(t_j), \nu^n(t_j))$ for all N paths.

Step 4: Pathwise repricing and statistical verification. For every path n and each intermediate month t_j , let $\tau_j = T - t_j$ denote the remaining maturity. The code then evaluates the continuation value $\mathcal{O}(S^n(t_j), \nu^n(t_j))$. The implementation reprices directly from each simulated state and the repricing is written in terms of the current normalized strike $\kappa_t = K/S_t$, so that normalized option values are first computed and then rescaled by S_t . The corresponding Monte Carlo averages and standard errors are then computed for the stock and each contract.

3.1 Time-0 Fourier Pricing

The time-0 option prices are computed analytically using the SINC Fourier pricing engine from CFLib [6]. Each vanilla option price is a linear combination of two digital prices, each of which is a probability under a different equivalent martingale measure. The SINC algorithm evaluates these probabilities by summing a cosine series that converges to the inverse Fourier transform of the characteristic function. Concretely, for normalized strike $\kappa_t = K/S_0$ and remaining time τ :

$$\mathcal{P}_{cn} = \mathcal{P}(X_\tau < \log(\kappa_t)) \quad \text{under } \mathbb{Q}$$

$$\mathcal{P}_{an} = \mathcal{P}(X_\tau < \log(\kappa_t)) \quad \text{under } \mathbb{S}$$

$$Std.Put = \kappa_t \cdot \mathcal{P}_{cn} - \mathcal{P}_{an}$$

$$Std.Call = Std.Put + (1 - \kappa_t)$$

where $\mathbb{Q}$ is the pricing measure and $\mathbb{S}$ is the stock numeraire measure. These quantities are computed once and serve as the benchmark values for the martingale test. The implementation also exposes a maturity-scaled cutoff mode, in which $X_c(\tau) = X_c\sqrt{\tau}$ rather than being held fixed. Its behavior is examined in Section 4.8.

3.2 Variance Simulation: Quadratic-Exponential Scheme

The CIR variance process is simulated using the Quadratic-Exponential (QE) scheme, implemented as `QT_cir_evol` in CFLib [4]. This scheme is required because the Feller condition is violated (ratio ≈ 0.1614): naive discretization can push the variance negative, which is inadmissible. The QE scheme works by matching the first two moments of the true chi-squared transition distribution of ν_t at each step. Depending on the conditional dispersion ratio,

$$\Psi = \frac{\text{Var}[\nu_t \mid \nu_{t-1}]}{\mathbb{E}[\nu_t \mid \nu_{t-1}]^2}$$

it switches between an exponential distribution (high dispersion) and a quadratic approximation to a Gaussian (low dispersion). This ensures that the variance remains non-negative throughout. The simulation also tracks the integrated variance

$$I_t = \int_0^t \nu_s ds$$

via the trapezoid rule at each sub-step, which is needed for exact spot evolution. The internal step $dt = 1/730$ (twice daily) is finer than the monthly output grid to ensure accuracy of the integrated variance.

3.3 Spot Price Simulation: Exact Conditional Scheme

Given the variance path $\{\nu_t\}$ and integrated variance $\{I_t\}$, the log-price increment over $[t_{j-1}, t_j]$ is computed by the function `mc_heston` using the exact conditional formula [5]:

$$\Delta X = -0.5 \Delta I + \left(\frac{\rho}{\eta} \right) (\Delta \nu - \lambda(\bar{\nu} \Delta t - \Delta I)) + \sqrt{(1 - \rho^2) \Delta I} \xi$$

where $\xi \sim \mathcal{N}(0, 1)$, independent of variance innovations. This formula is exact conditional on the variance path and correctly decomposes the correlated Brownian component ρdY_t into the observable variance innovation $\Delta \nu$ and the independent residual $\sqrt{1 - \rho^2} \xi$. No discretization bias is introduced in the spot path beyond that of the variance simulation.

3.4 Vectorized Pathwise Fourier Repricing

At each monitoring date t_j , for every simulated path n , the code evaluates the continuation value of all six claims at the state $(S^n(t_j), \nu^n(t_j))$ with residual maturity. A brute-force nested Monte Carlo approach would require simulating additional inner paths from each intermediate state. This is unnecessary here because the Heston pricing problem admits an analytical-numerical solution via the characteristic function, which remains available in affine form at any intermediate state. Since $\log \varphi(k, \tau, \nu_t) = A(k, \tau) - \nu_t B(k, \tau)$, the complex functions A and B are identical for all N paths at a given time step. For each Fourier mode m , A and B are computed once as complex scalars, and the characteristic function for all N paths is then evaluated simultaneously via

$$\varphi^n(k, \tau) = \exp(A(k, \tau) - \nu^n(t_j) B(k, \tau))$$

which is implemented through NumPy broadcasting over the N variance values. This reduces the SINC loop from $\mathcal{O}(N \times M)$ scalar characteristic function evaluations to $\mathcal{O}(M)$ evaluations with N -dimensional array arithmetic at each mode, where M is the number of Fourier modes required for convergence (here between 255 and 2445, depending on residual maturity; see Section 4.7). The normalized strikes

$$\kappa^n = \frac{K}{S^n(t_j)}$$

and log-moneyness values are also computed as vectors, so each cosine and sine evaluation operates over all N paths simultaneously. The final option values are then rescaled by the path-specific forward $S^n(t_j)$. This approach is not only computationally efficient but also conceptually correct. This reprices each path from the full Markov state (S^n, ν^n) rather than from the stock price alone, thereby honoring the bivariate state structure of the Heston model.

4 Numerical results and discussion

4.1 Time-0 benchmark prices

The first set of numerical results consists of the time-0 prices of the six claims computed by the Fourier engine. These constitute the benchmark levels for the entire martingale verification. Table 2 reports the benchmark values obtained.

Table 2: Time-0 prices under the Heston model.

Claim	Time-0 price	Notes
S_0	1.2	Initial stock price
Put	0.082958	= Call by put-call parity (ATM, $r = q = 0$)
Call	0.082958	Confirmed: $\mathcal{C} - \mathcal{P} = S_0 - K = 0$
$\mathcal{P}_{cn}$	0.392848	$\mathcal{P}(S_T < K) \approx 39.3\%$; below 50% due to $\rho < 0$
$\mathcal{P}_{an}$	0.388460	$\mathbb{E}[S_T \mathbf{1}_{[K > S_T]}]$; asset-weighted probability
$\mathcal{C}_{cn}$	0.607152	= $1 - \mathcal{P}_{cn}$; $\mathcal{P}(S_T > K) \approx 60.7\%$
$\mathcal{C}_{an}$	0.811540	= $S_0 - \mathcal{P}_{an} = 1.2 - 0.389 = 0.812$

Although the option is at-the-money ($S_0 = K = 1.2$) and Put = Call by parity, the digital prices are not symmetric around one half. $\mathcal{P}_{cn} \approx 0.393 < 0.5$ and $\mathcal{C}_{cn} \approx 0.607 > 0.5$. This asymmetry is consistent with the negative stock-variance correlation $\rho = -0.645$ (the leverage effect).

4.2 Internal consistency checks

Before analyzing the martingale diagnostics, it is useful to verify that the time-0 results satisfy the main pricing identities implied by the theoretical framework. First, put-call parity

$$\mathcal{C}(S_0, T) - \mathcal{P}(S_0, T) = S_0 - K = 0$$

is confirmed numerically by the equality of the call and put prices reported in Table 2. Second, the two cash-or-nothing claims must exhaust the terminal event space and third, the two asset-or-nothing claims must split the expected terminal stock value, so that

$$\mathcal{P}_{cn}(S_0, T) + \mathcal{C}_{cn}(S_0, T) = 1$$

$$\mathcal{P}_{an}(S_0, T) + \mathcal{C}_{an}(S_0, T) = S_0$$

Numerically,

$$0.392848 + 0.607152 = 1.0$$

$$0.38846 + 0.81154 = 1.2 = S_0$$

These checks confirm that the pricing layer of the implementation is internally coherent, and they provide a first validation that the claims are being computed consistently with the payoff decomposition discussed in Section 2. The martingale experiment is therefore a dynamic extension of this same consistency logic from time 0 to the intermediate monitoring dates.

4.3 Monthly martingale diagnostics for Stock, Put and Call

Table 3 reports the monthly Monte Carlo means and standard errors for the stock, the put continuation value, and the call continuation value. The benchmark values remain constant through time and are equal to the corresponding time-0 prices in Table 2. Hence, the martingale property is numerically verified when the monthly Monte Carlo means remain close to those benchmark levels and the deviations are of the same order as the reported sampling error.

Table 3: Monthly martingale diagnostics for S_t , Put, and Call.

t_j	$\mathbb{E}[S_{t_j}]$	SE	$\mathbb{E}[V_{t_j}^P]$	SE	$\mathbb{E}[V_{t_j}^C]$	SE
0.083333	1.199301	0.000760	0.083369	0.000541	0.082671	0.000369
0.166667	1.198595	0.001082	0.083918	0.000779	0.082513	0.000505
0.250000	1.199107	0.001327	0.083828	0.000960	0.082936	0.000620
0.333333	1.199467	0.001521	0.083934	0.001098	0.083402	0.000711
0.416667	1.198558	0.001698	0.084253	0.001227	0.082811	0.000788
0.500000	1.199023	0.001852	0.083822	0.001337	0.082844	0.000864
0.583333	1.199858	0.001988	0.082943	0.001430	0.082801	0.000934
0.666667	1.200615	0.002132	0.082444	0.001517	0.083060	0.001030
0.750000	1.201347	0.002262	0.082075	0.001601	0.083423	0.001106
0.833333	1.200568	0.002368	0.082313	0.001687	0.082881	0.001141
0.916667	1.200689	0.002482	0.082311	0.001767	0.083000	0.001203

The stock process remains very close to its theoretical benchmark $S_0 = 1.2$ throughout the monitoring horizon, with no persistent upward or downward drift. The vanilla put and call continuation values similarly fluctuate around the common benchmark 0.082958 without systematic bias. Standard errors grow monotonically as t_j increases, which is expected: at later monitoring dates, the continuation values are computed from simulated states that are further from the initial condition and therefore exhibit greater cross-sectional variability.

4.4 Monthly martingale diagnostics for digital and asset-or-nothing claims

Table 4 extends the diagnostics to the four indicator-based claims.

Table 4: Monthly martingale diagnostics for $\mathcal{P}_{cn}$, $\mathcal{P}_{an}$, $\mathcal{C}_{cn}$, and $\mathcal{C}_{an}$.

t_j	$\mathbb{E}[V_{t_j}^{\mathcal{P}_{cn}}]$	SE	$\mathbb{E}[V_{t_j}^{\mathcal{P}_{an}}]$	SE	$\mathbb{E}[V_{t_j}^{\mathcal{C}_{cn}}]$	SE	$\mathbb{E}[V_{t_j}^{\mathcal{C}_{an}}]$	SE
0.083333	0.394096	0.001696	0.389546	0.001582	0.605904	0.001696	0.809756	0.002308
0.166667	0.394477	0.002229	0.389455	0.002059	0.605523	0.002229	0.809141	0.003048
0.250000	0.393471	0.002584	0.388336	0.002392	0.606529	0.002584	0.810771	0.003553
0.333333	0.392708	0.002870	0.387315	0.002662	0.607292	0.002870	0.812152	0.003957
0.416667	0.395008	0.003157	0.389757	0.002947	0.604992	0.003157	0.808801	0.004356
0.500000	0.394292	0.003410	0.389329	0.003209	0.605708	0.003410	0.809694	0.004707
0.583333	0.393639	0.003664	0.389424	0.003483	0.606361	0.003664	0.810434	0.005055
0.666667	0.393559	0.003919	0.389826	0.003762	0.606441	0.003919	0.810789	0.005412
0.750000	0.393120	0.004186	0.389669	0.004058	0.606880	0.004186	0.811679	0.005775
0.833333	0.392518	0.004457	0.388709	0.004358	0.607482	0.004457	0.811859	0.006124
0.916667	0.393426	0.004781	0.389801	0.004747	0.606574	0.004781	0.810889	0.006550

These contracts are more sensitive to simulation noise because their payoffs involve either a discontinuous indicator or an indicator-weighted stock value, both of which have higher variance than vanilla option payoffs. The digital and asset-or-nothing contracts display larger standard errors than the vanilla options. Nevertheless, the monthly averages continue to fluctuate around the theoretical benchmarks without systematic bias, and the observed deviations remain compatible with Monte Carlo uncertainty. These contracts are more sensitive to the shape of terminal distribution and their stability provides additional evidence that the state-dependent repricing is implemented correctly.

4.5 Graphical martingale verification

Figure 1 summarizes the martingale verification. In each panel, the red line represents the time-0 benchmark, and the green markers with error bars are the monthly Monte Carlo estimates.

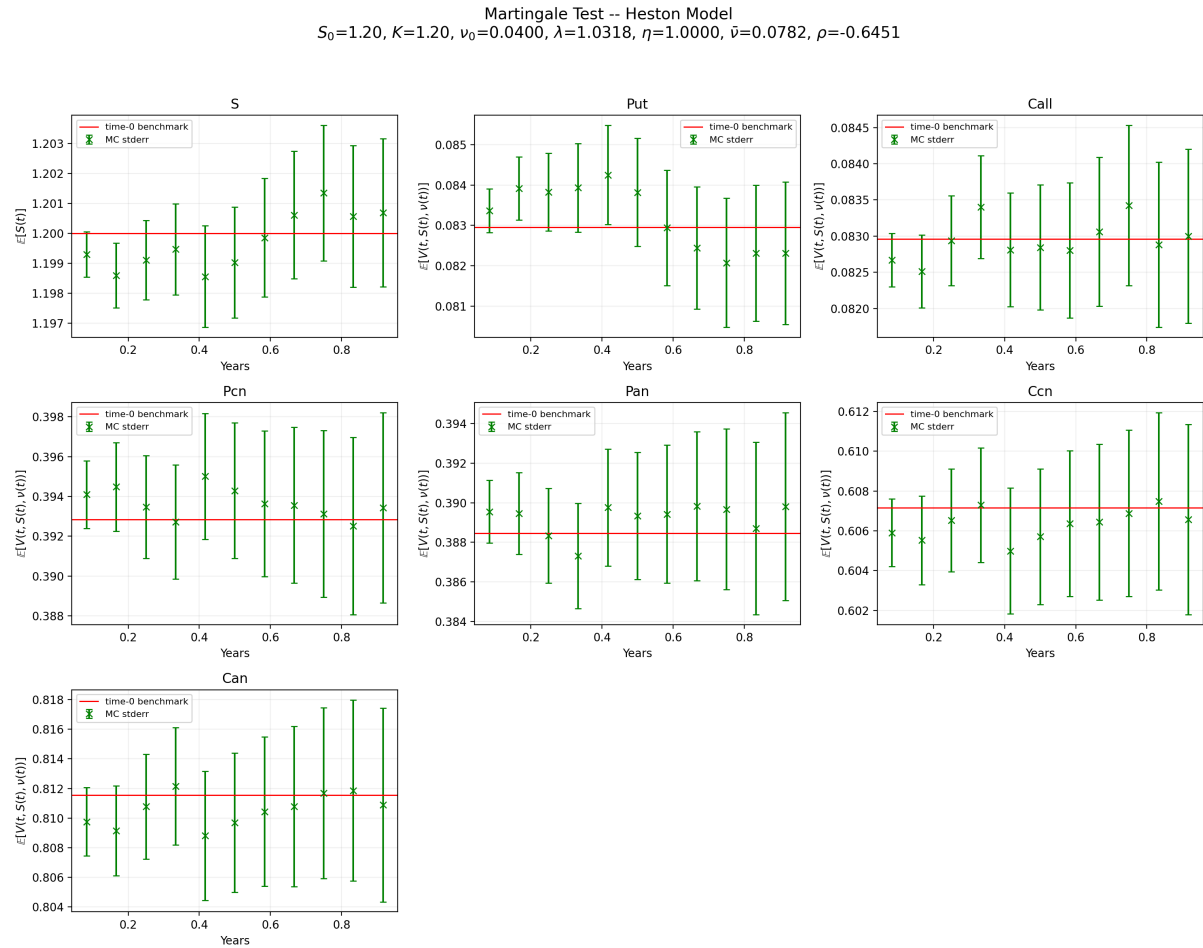


Figure 1: Martingale diagnostics for the stock and all six claims.

Figure 1 allows the behavior of all seven quantities to be assessed simultaneously. The stock process appears visually stable around its benchmark level, and the same holds for the continuation values of all six contracts. There is no evidence of systematic drift, no persistent bias away from the red benchmark line, and no sign that any particular payoff class behaves inconsistently with the martingale condition. Instead, the deviations appear random and centered, which is exactly the pattern expected from a successful Monte Carlo martingale check.

4.6 Maximum deviation analysis

A useful summary statistic is obtained by comparing the absolute deviation from the benchmark with the corresponding standard error:

$$\max_j \frac{|\widehat{\mathbb{E}}[Z_{t_j}] - Z_0|}{\text{SE}(\widehat{\mathbb{E}}[Z_{t_j}])}$$

Table 5: Maximum absolute deviation from the benchmark, measured in standard errors.

Quantity	Max deviation/SE	Date of maximum (t)
S_t	1.298	0.1667
Put	1.231	0.1667
Call	0.882	0.1667
$\mathcal{P}_{cn}$	0.735	0.0833
$\mathcal{P}_{an}$	0.687	0.0833
$\mathcal{C}_{cn}$	0.735	0.0833
$\mathcal{C}_{an}$	0.787	0.1667

Table 5 reports the quantity for each claim. The largest observed deviation across all monitored dates and all seven quantities is 1.3 standard errors. This is entirely consistent with ordinary Monte Carlo fluctuations at the $N = 8,192$ sample size, and far below the conventional threshold of 1.96 standard errors used for hypothesis testing at the 5% level. The increase in error bars over time reflects greater cross-sectional variability in continuation values near maturity and does not indicate a failure of the martingale property, but is instead an expected feature of the Monte Carlo estimator.

4.7 Computational cost

Section 3.4 asserts that vectorized repricing is more efficient than nested simulation; this section quantifies that claim. Table 6 reports the wall-clock cost of each stage for the baseline configuration $N = 2^{13} = 8192$, measured on an Apple M1 MacBook Air running macOS 26 under Python 3.13 with NumPy.

Table 6: Wall-clock cost by stage, $N = 2^{13} = 8192$, seed 29283

Stage	Time (s)	Share
Time-0 Fourier pricing (6 claims)	0.002	0.03%
Outer path simulation (variance + spot)	0.504	8.1%
Vectorized repricing loop (11 dates)	5.016	80.4%
Output generation (tables, CSV, figure)	0.718	11.5%
Total	6.240	100%

The repricing loop dominates at roughly 80% of total runtime while path generation accounts

for only 8%: the cost of the experiment is concentrated in valuation, not simulation. Time-0 pricing of all six claims costs under two milliseconds, which is what makes the Fourier engine viable as a repricing primitive rather than merely as a one-off benchmark.

The repricing cost is governed by the number of SINC modes M required to reach the convergence tolerance of 10^{-8} , and this is far from constant across the monitoring grid. Instrumenting the SINC loop shows M growing from approximately 255 terms at $\tau = 0.9167$ to approximately 2445 terms at $\tau = 0.0833$, scaling roughly as $1/\tau$. The reason is structural: as the residual maturity shrinks, the conditional density of X_τ becomes increasingly peaked, and resolving a narrow density against a fixed truncation range X_c requires proportionally more modes. The later monitoring dates are therefore an order of magnitude more expensive than the early ones.

Comparison with nested Monte Carlo. The alternative to state-dependent Fourier repricing is nested simulation: at each of the 11 intermediate dates, and for each of the $N = 8192$ outer states, an independent bundle of inner trajectories would be run from $(S^n(t_j), \nu^n(t_j))$ forward to maturity. Even with a modest inner sample of $N_{\text{inner}} = 10^3$, this requires $11 \times 8192 \times 10^3 \approx 9.0 \times 10^7$ inner trajectories. Benchmarking the inner simulation directly gives approximately 27.5 ms per outer state, so the complete nested calculation would require on the order of 2.5×10^3 s, roughly 41 minutes: close to 490 times the cost of the vectorized loop it replaces, by the mechanism set out in Section 3.4. The saving is statistical as well as computational, since a nested estimator would superimpose inner-sampling noise on the outer-path variance and inflate the standard errors reported in Tables 3 and 4.

4.8 Sensitivity to the Fourier cutoff

Because the number of SINC modes grows sharply as $\tau \rightarrow 0$, a natural optimization is to shrink the truncation range with the residual maturity, setting $X_c(\tau) = X_c\sqrt{\tau}$ so that the cutoff tracks the width of the log-price distribution. The script supports this through the `-xcmode sqrt_tau` switch.

Empirically, the saving is real, but the accuracy cost is not acceptable here. Summed over all 22 SINC calls in the monitoring loop, the fixed cutoff requires 16,255 terms against 8,829 for the scaled cutoff, a reduction of 46%. However, the scaled cutoff truncates the density too aggressively at short residual maturity and biases the continuation values downward, as shown in Table 7.

Table 7: Effect of the cutoff mode on $\mathbb{E}[V_t^P]$. Benchmark: $P(S_0, T) = 0.082958$.

τ	fixed	sqrt_tau	benchmark
0.9167	0.083369	0.083368	0.082958
0.2500	0.082075	0.081781	0.082958
0.1667	0.082313	0.081440	0.082958
0.0833	0.082311	0.078072	0.082958

At $\tau = 0.0833$ the scaled cutoff understates the put continuation value by 5.9%, approximately 2.8 standard errors, which is a systematic bias rather than sampling noise; the fixed cutoff remains within 1.3 standard errors at every date. All results reported elsewhere in this document therefore use $X_c = 5.0$ held fixed. Recovering the speed advantage safely would require a larger base cutoff paired with the $\sqrt{\tau}$ scaling, which was not pursued here.

5 Interpretation of the results

1. **Stock process.** The sample mean of S_t remains very close to $S_0 = 1.2$ at all monthly dates. There is no systematic drift in either direction. The standard errors grow as approximately $\sqrt{t_j}$, which is consistent with the diffusive nature of the log-price under the Heston model. So, the simulation preserves the risk-neutral martingale property of the stock price process.
2. **Vanilla options.** Both Put and Call continuation values fluctuate around the common benchmark. The error bars for the call are slightly narrower than for the put, which reflects lower Monte Carlo variability. The put-call symmetry is preserved throughout.
3. **Cash-or-nothing digitals: $\mathcal{P}_{cn}$ and $\mathcal{C}_{cn}$.** These contracts exhibit larger standard errors than vanilla options, reflecting the high variance of binary payoffs. The complementary symmetry is preserved throughout the experiment: when $\mathcal{P}_{cn}$ is above its benchmark, $\mathcal{C}_{cn}$ is below by exactly the same amount, since $\mathcal{P}_{cn} + \mathcal{C}_{cn} = 1$ holds pathwise and also in expectation. Both hover around their benchmarks without systematic bias.
4. **Asset-or-nothing digitals: $\mathcal{P}_{an}$ and $\mathcal{C}_{an}$.** The asset-or-nothing claims inherit additional variance from the state-dependent weight S_t , resulting in the widest error bars among all contracts. $\mathcal{P}_{an} \approx 0.389$ is slightly smaller than $\mathcal{P}_{cn} \approx 0.393$ because the indicator is weighted by S_T , which downweights low-stock states. $\mathcal{C}_{an} = S_0 - \mathcal{P}_{an} \approx 0.812$ is the largest value among six option prices. Both remain centered around their benchmarks throughout.
5. **Widening error bars over time.** It is natural that the error bars widen as t_j increases. At later monitoring dates, the continuation values are evaluated from simulated states $(S^n(t_j), \nu^n(t_j))$ that are further from the initial condition and more dispersed across paths. Also, the shorter residual maturity means that the option value at each state is more sensitive to the local variance level $\nu^n(t_j)$. The increasing cross-sectional variability of continuation values over time is a structural feature of the Monte Carlo experiment.

6 Limitations and possible extensions

The experiment confirms the martingale property under one parameter set and one numerical configuration. Several limitations are worth stating explicitly.

1. **Martingale correction in the QE scheme.** The quadratic-exponential discretization is not exactly martingale-preserving for the spot process. Andersen [4] proposes a martingale correction that enforces $\mathbb{E}[S_{t_{j+1}} | \mathcal{F}_{t_j}] = S_{t_j}$ exactly at each step. It is not applied here. No residual discretization drift is detectable at $N = 2^{13}$, but absence of detection is not absence of bias: the standard error shrinks as $1/\sqrt{N}$ while a discretization bias does not, so the same experiment at larger sample sizes would eventually resolve one if present.
2. **Violated Feller condition.** The parameter set gives $2\lambda\bar{\nu}/\eta^2 \approx 0.1614$, far below unity, so the variance process reaches zero with non-negligible probability. The QE scheme handles this without producing negative variances, but the accuracy of any variance discretization degrades in this regime. A comparison against the exact scheme of Broadie and Kaya [5] would isolate the discretization error from the sampling error.

3. **Single parameter set.** Only one calibration is tested. The leverage effect is strong ($\rho = -0.645$) and the option is exactly at-the-money, which is a comparatively favorable configuration. Deep out-of-the-money strikes, where digital payoffs become rare events, would stress the estimator considerably harder and would likely require importance sampling.
4. **No variance reduction.** The estimator is plain Monte Carlo. Antithetic variates on the independent Brownian component, or a control variate built from the Black-Scholes price at the realized integrated variance, would reduce the standard errors materially at no additional simulation cost, particularly for the indicator-based claims whose error bars are widest.
5. **European claims only.** The vectorized repricing exploits the fact that continuation values of European claims are available in closed Fourier form at any intermediate state. Path-dependent claims admit no such representation, and verifying the martingale property for a barrier or Asian contract would require either nested simulation or a regression-based continuation-value estimator.

7 Conclusion

This report presents a complete implementation and numerical verification of the martingale property under the Heston model. The workflow combined time-0 Fourier pricing, Monte Carlo simulation of monthly Heston states, and pathwise Fourier repricing at intermediate dates to verify the martingale property for the stock and six European-type claims. All quantities remained within the ordinary Monte Carlo error of their time-0 benchmarks. The resulting numerical evidence is fully consistent with the martingale property required. Both the tables and the graphical diagnostics confirm that

$$\mathbb{E}[S_t] = S_0, \quad \mathbb{E}\left[V^{\mathcal{O}}(t, S_t, \nu_t)\right] = \mathcal{O}(S_0, T)$$

holds up to Monte Carlo statistical error for all claims considered. This confirms the internal consistency of the Fourier pricing engine, the stochastic volatility simulation, and the intermediate-date valuation procedure under the Heston model.

A Module Layout

The solution is a single self-contained Python script `heston_martingales.py` that imports from the `CFLib` library. For completeness, implemented functions and their roles are summarized below:

Table 8: Code structure and function roles.

Function	Role
<code>parse_inputs()</code>	Reads numerical parameters; enforces $r = q = 0$
<code>build_model()</code>	Instantiates the Heston object from <code>CFLib</code>
<code>x_cutoff()</code>	Returns fourier cutoff X_c (fixed = 5 or <code>sqrt_tau</code> mode)
<code>option_prices_t0()</code>	Computes time-0 Fourier prices using <code>CFLib.ft_opt</code>
<code>_cf_components(model,k,tau)</code>	Extracts the complex scalars from the Heston CF
<code>_pr_x_lt_w_vec()</code>	Vectorized SINC loop: evaluates $P(X < w)$ for all N paths
<code>ft_opt_vec()</code>	Option suite via vectorized SINC; returns normalized prices
<code>option_prices_vec()</code>	Scales normalized outputs back to actual prices path-by-path
<code>build_schedule()</code>	Monthly time grid $[0, 1/12, 2/12, \dots, 1]$
<code>simulate_outer_paths()</code>	Uses <code>QT_cir_evol</code> for variance and <code>mc_heston</code> for spot
<code>martingale_test()</code>	Loops over 11 time steps; records MC means and SEs
<code>main()</code>	Orchestrates all stages; times each phase

A.1 Execution Instructions and Output Files

The repository contains the main script `heston_martingales.py` together with the required `CFLib` dependency. Numerical results are printed to the terminal during execution for immediate inspection. The repository includes a pre-generated set of outputs in `./heston_outputs`; re-running the script regenerates them in place. The directory is created automatically upon execution. In particular:

- `martingale_diagnostics.png`: the full set of martingale diagnostic plots
- `heston_summary.csv`: Monte Carlo estimates, standard errors and benchmark values

References

- [1] S. L. Heston. A closed-form solution for options with stochastic volatility with applications to bond and currency options. *The Review of Financial Studies*, 6(2): 327-343, 1993.
- [2] J. C. Cox, J. E. Ingersoll, and S. A. Ross. A theory of the term structure of interest rates. *Econometrica*, 53(2): 385-407, 1985.
- [3] W. Feller. Two singular diffusion problems. *Annals of Mathematics*, 54(1): 173-182, 1951.
- [4] L. B. G. Andersen. Simple and efficient simulation of the Heston stochastic volatility model. *Journal of Computational Finance*, 11(3): 1-42, 2008.
- [5] M. Broadie and Ö. Kaya. Exact simulation of stochastic volatility and other affine jump diffusion processes. *Operations Research*, 54(2): 217-231, 2006.
- [6] F. Baschetti, G. Bormetti, S. Romagnoli, and P. Rossi. The SINC way: a fast and accurate approach to Fourier pricing. *Quantitative Finance*, 22(3): 427-446, 2022.